Raspberry Pi 4 Lab Experiments (EXP 08 - 12)

Short codes, connections and run steps

EXP 08: GPIO LED

Connection

LED	Raspberry Pi
Long leg (+) via 220 ohm resistor	GPIO 17 (pin 11)
Short leg (-)	GND (pin 6)

Code (led.py)

```
from gpiozero import LED
from time import sleep

led = LED(17)
while True:
    led.on();  sleep(1)
    led.off(); sleep(1)
```

Run: python3 led.py **Stop:** Ctrl + C

Remember: easiest one, uses gpiozero. If missing: sudo apt install python3-gpiozero

EXP 09: Ultrasonic Sensor (HC-SR04)

Connection

HC-SR04	Raspberry Pi
VCC	5V (pin 2)
GND	GND (pin 6)
TRIG	GPIO 23 (pin 16)
ECHO	GPIO 24 (pin 18) via voltage divider

Voltage divider: ECHO - 1k ohm - junction - GPIO 24; junction - 2k ohm - GND. This drops 5V to about 3.3V to protect the Pi.

Code (distance.py)

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setup(23, GPIO.OUT)
GPIO.setup(24, GPIO.IN)

try:
    while True:
        GPIO.output(23, 1)
        time.sleep(0.00001)
        GPIO.output(23, 0)

        while GPIO.input(24) == 0:
            start = time.time()
        while GPIO.input(24) == 1:
            end = time.time()

        print("Distance:", round((end - start) * 17150, 2), "cm")
        time.sleep(1)
except KeyboardInterrupt:
    GPIO.cleanup()
```

Run: python3 distance.py
Remember: distance = time x 17150 (speed of sound / 2). TRIG = output, ECHO = input. If no echo returns, this simple version can hang.

EXP 10: IR Sensor

Connection

IR Sensor	Raspberry Pi
VCC	3.3V (pin 1)
GND	GND (pin 6)
OUT	GPIO 17 (pin 11)

Code (ir.py)

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setup(17, GPIO.IN)

while True:
    if GPIO.input(17) == 0:
        print("Object Detected")
    else:
        print("No Object")
    time.sleep(1)
```

Run: python3 ir.py
Remember: output is LOW (0) when an object is detected. If your module behaves oppositely, swap the messages. The small potentiometer on the module adjusts the detection range. An IR obstacle sensor only detects presence, not exact distance.

EXP 11: Relay + Buzzer / LED

Relay module to Raspberry Pi

Relay pin	Raspberry Pi
VCC	5V (pin 2)
GND	GND (pin 6)
IN	GPIO 27 (pin 13)

Load side (LED)

From	Connect to
External DC supply (+)	Relay COM
Relay NO	220 ohm resistor
Resistor other end	LED long leg (+)
LED short leg (-)	External supply (-)
Relay NC	Leave empty

Buzzer: same wiring, but connect the buzzer (+) to NO and (-) to supply GND, with no resistor, if it matches the supply voltage.
Important: never connect the LED directly to GPIO 27 or the relay VCC. It must sit in the switched circuit between COM and NO.

Code (relay_led.py)

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setup(27, GPIO.OUT)

try:
    while True:
        GPIO.output(27, GPIO.LOW)   # Relay ON
        print("ON")
        time.sleep(2)
        GPIO.output(27, GPIO.HIGH)  # Relay OFF
        print("OFF")
        time.sleep(2)
except KeyboardInterrupt:
    pass
finally:
    GPIO.cleanup()
```

Run: python3 relay_led.py
Remember: most relay modules are active-low (LOW = ON). If yours is active-high, swap LOW and HIGH.

EXP 12: Client-Server (Socket)

server.py (run on the Raspberry Pi)

```
import socket

s = socket.socket()
s.bind(("0.0.0.0", 5000))
s.listen(1)
conn, addr = s.accept()
print("Connected:", addr)

while True:
    data = conn.recv(1024).decode()
    if not data or data == "exit":
        break
    print("Client:", data)
    conn.send("Received".encode())

conn.close()
```

client.py (run on the other device)

```
import socket

c = socket.socket()
c.connect(("192.168.1.100", 5000))   # Pi's IP

while True:
    msg = input("Enter: ")
    c.send(msg.encode())
    if msg == "exit":
        break
    print("Server:", c.recv(1024).decode())

c.close()
```

Steps

• On the Pi, run **hostname -I** to get its IP and put it in client.py.
• Run **python3 server.py** first, then **python3 client.py**.
• Both devices must be on the same network.

Quick Memory Table

Exp	Key idea	Remember
08 LED	LED(17), on/off	Easiest, uses gpiozero
09 Ultrasonic	TRIG out, ECHO in	Distance = time x 17150
10 IR	Input pin, 0 = detected	Just read and print
11 Relay	Output pin, LOW = ON	Load goes through COM + NO
12 Socket	Server: bind, listen, accept. Client: connect	Server first, then client

Common pattern for 09, 10, 11: setmode(BCM) - setup(pin, IN/OUT) - loop - cleanup()

Pin check: always match your lab manual if it specifies different pins.